LA CITE – SCIENCES DES DONNEES APPLIQUEES

CONCEPTION ET MODÉLISATION D'UNE BASE DE DONNÉES MONGODB - SYSTÈME DE RÉSERVATION DE SALLES DE COWORKING

Introduction aux bases de données massives

Misse, Anthony	2753251
Mutanda, Josue Mulumba	2751096
Talling Djafa, Channel Borris	2751491
Zahabi, Sherine	2711188

Mars 2026

TABLE DES MATIERES

1. INTRODUCTION 2

1.1 CONTEXTE ET PRÉSENTATION DU PROJET 2
1.2 OBJECTIFS DU PROJET 2
1.3 OUTILS UTILISÉS 2

2 MODÉLISATION ENTITÉ-ASSOCIATION 3

2.1 IDENTIFICATION DES ENTITÉS 3
2.2 DESCRIPTION DES ATTRIBUTS 3
2.3 RELATIONS ET CARDINALITÉS 5
2.4 DIAGRAMME E/A 6

3 ANALYSE DES CHARGES 6

3.1 BESOINS FONCTIONNELS DU SYSTÈME 6
3.2 FRÉQUENCE DES REQUÊTES 7
3.3 VOLUME DES DONNÉES ESTIMÉ 7
3.4 PATTERNS D'ACCÈS IDENTIFIÉS 8

4 CONCEPTION — DESIGN PATTERNS MONGODB 8

4.1 VUE D'ENSEMBLE DES PATTERNS RETENUS 8
4.2 PATTERN 1 — EMBEDDING : ESPACES_COWORKING 9
4.3 PATTERN 2 — EMBEDDING + SUBSET + COMPUTED : SALLES 10
4.4 PATTERN 3 — EXTENDED REFERENCE : RESERVATIONS 11
4.5 PATTERN 4 — COLLECTIONS SÉPARÉES : UTILISATEURS ET AVIS 11
4.6 SCHÉMA FINAL DES COLLECTIONS 12

5 IMPLÉMENTATION 12

5.1 CRÉATION DE LA BASE DE DONNÉES ET DES COLLECTIONS 13
5.2 INSERTION DES DONNÉES DE TEST 13
5.3 VUE DES COLLECTIONS DANS MONGODB COMPASS 15

6 EXÉCUTION DES REQUÊTES CRUD 15

6.1 REQUÊTE 1 — RECHERCHE DE SALLES DISPONIBLES 15
6.2 REQUÊTE 2 — VÉRIFICATION DE CRÉNEAU ET CRÉATION DE RÉSERVATION 16
6.3 REQUÊTE 3 — ANNULATION D'UNE RÉSERVATION AVEC REMBOURSEMENT 17
6.4 REQUÊTE 4 — AJOUT D'UN AVIS ET MISE À JOUR DE LA SALLE 17
6.5 REQUÊTE 5 — HISTORIQUE DES RÉSERVATIONS D'UN UTILISATEUR 18

7 CONCLUSION 19

7.1 BILAN DES CHOIX DE MODÉLISATION 19
7.2 AVANTAGES DE MONGODB PAR RAPPORT À UN MODÈLE RELATIONNEL 19
7.3 LIMITES ET AMÉLIORATIONS POSSIBLES 20

1. INTRODUCTION

1.1 CONTEXTE ET PRÉSENTATION DU PROJET

Dans un contexte où le travail flexible et le télétravail sont en pleine expansion, les espaces de coworking connaissent une croissance significative. Ces lieux partagés offrent à des professionnels, freelances et équipes la possibilité de louer des espaces de travail à la demande, qu'il s'agisse de bureaux privés, de salles de réunion ou d'espaces ouverts.

Ce projet consiste à concevoir et implémenter une base de données pour une **plateforme de réservation de salles de coworking**. Cette plateforme permet à des utilisateurs de consulter les salles disponibles, d'effectuer des réservations sur des créneaux horaires définis, de procéder au paiement et de laisser des avis sur les espaces utilisés. Elle offre également aux administrateurs la gestion des espaces, des salles et des équipements disponibles.

1.2 OBJECTIFS DU PROJET

Ce projet vise à mettre en pratique les concepts fondamentaux des bases de données orientées documents à travers les objectifs suivants :

- **Modéliser** les données du système sous forme d'un diagramme Entité-Association (E/A) en définissant les entités, leurs attributs, les relations et les cardinalités
- **Analyser les charges** du système afin d'identifier les besoins en lecture et en écriture, le volume des données et les patterns d'accès prioritaires
- **Concevoir** la structure des collections MongoDB en appliquant des design patterns adaptés (Embedding, Extended Reference, Subset Pattern, Computed Pattern) et en justifiant chaque choix
- **Implémenter** la base de données avec ses collections, sa validation par schéma ($jsonSchema) et ses index d'optimisation
- **Interroger** la base de données à l'aide de requêtes CRUD variées couvrant les opérations de lecture, d'écriture, de mise à jour et d'agrégation

1.3 OUTILS UTILISÉS

Outil	Version	Rôle
MongoDB Community Server	7.0	Moteur de base de données NoSQL orienté documents
MongoDB Compass	Dernière version stable	Interface graphique pour visualiser et explorer les collections
Visual Studio Code	Dernière version stable	Éditeur de code pour la rédaction des scripts *.mongodb.js*
Extension MongoDB for VS Code	Dernière version stable	Exécution des scripts directement depuis l'éditeur avec connexion à la base locale
mongosh	Intégré à Compass	Shell JavaScript interactif pour l'exécution des requêtes

Le choix de **MongoDB** se justifie par la nature des données du projet : les entités telles que les salles (avec leurs équipements et leurs avis), les réservations (avec leurs paiements) et les espaces coworking (avec leurs adresses et horaires) présentent des structures imbriquées et hiérarchiques qui se modélisent naturellement sous forme de documents JSON. MongoDB permet ainsi d'éviter les jointures coûteuses d'un modèle relationnel et d'optimiser les performances en lecture pour les requêtes les plus fréquentes.

2 MODÉLISATION ENTITÉ-ASSOCIATION

La modélisation Entité-Association constitue la première étape de conception de la base de données. Elle permet d'identifier les entités du système, leurs attributs et les relations qui les lient, indépendamment de toute technologie de stockage.

2.1 IDENTIFICATION DES ENTITÉS

Le système de réservation de salles de coworking repose sur **10 entités** distinctes :

Entité	Description	Attribut clé
UTILISATEUR	Personne inscrite sur la plateforme pouvant effectuer des réservations	email (unique)
ESPACE_COWORKING	Site physique regroupant plusieurs salles disponibles à la réservation	nom
ADRESSE	Localisation géographique d'un espace coworking, incluant des coordonnées GPS	ville + code_postal
HORAIRES_OUVERTURE	Plage horaire d'ouverture d'un espace pour un jour donné de la semaine	jour
SALLE	Espace individuel réservable au sein d'un espace coworking	nom + type
EQUIPEMENT	Matériel ou service disponible dans une ou plusieurs salles	nom
SALLE_EQUIPEMENT	Entité associative résolvant la relation M:N entre salles et équipements	quantite
RESERVATION	Enregistrement de l'occupation d'une salle par un utilisateur sur une plage horaire	date_debut + date_fin
PAIEMENT	Transaction financière associée à une réservation	reference_transaction
AVIS	Évaluation laissée par un utilisateur sur une salle après utilisation	note

2.2 DESCRIPTION DES ATTRIBUTS

2.2.1 UTILISATEUR

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
nom	String	Obligatoire	Nom de famille
prenom	String	Obligatoire	Prénom
email	String	Unique, Obligatoire	Adresse email de connexion
telephone	String	Optionnel	Numéro de contact
mot_de_passe	String	Obligatoire	Mot de passe hashé (bcrypt)
type	Enum	Obligatoire	Rôle : membre, visiteur, admin
date_inscription	Date	Auto-générée	Date de création du compte

2.2.2 ESPACE_COWORKING

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
nom	String	Obligatoire	Nom commercial de l'espace
telephone	String	Optionnel	Numéro de contact de l'espace
description	String	Optionnel	Présentation générale de l'espace

2.2.3 ADRESSE

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
rue	String	Obligatoire	Nom de la rue
numero	String	Obligatoire	Numéro de la rue
ville	String	Obligatoire	Ville de localisation
code_postal	String	Obligatoire	Code postal
province	String	Optionnel	Province ou région
pays	String	Obligatoire	Pays (défaut : Canada)
complement	String	Optionnel	Informations complémentaires (étage, bureau…)

2.2.4 HORAIRES_OUVERTURE

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
jour	Enum	Obligatoire	Jour de la semaine (lundi → dimanche)
heure_ouverture	String	Optionnel	Heure d'ouverture au format HH:MM
heure_fermeture	String	Optionnel	Heure de fermeture au format HH:MM
ferme	Boolean	Obligatoire	Indique si l'espace est fermé ce jour-là

2.2.5 SALLE

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
nom	String	Obligatoire	Nom ou numéro de la salle
capacite	Integer	Obligatoire	Nombre maximum de personnes
type	Enum	Obligatoire	reunion, bureau_prive, espace_ouvert, cabine
superficie_m2	Float	Optionnel	Surface en mètres carrés
prix_heure	Float	Obligatoire	Tarif horaire en dollars
prix_journee	Float	Optionnel	Tarif journalier en dollars
disponible	Boolean	Obligatoire	Indique si la salle est active et réservable

2.2.6 EQUIPEMENT

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
nom	String	Obligatoire	Nom de l'équipement (ex : projecteur, tableau blanc)
type	Enum	Optionnel	technologique, mobilier, service
description	String	Optionnel	Détails supplémentaires sur l'équipement

2.2.7 SALLE_EQUIPEMENT

Attribut	Type	Contrainte	Description
quantite	Integer	Obligatoire	Nombre d'unités de l'équipement dans la salle

2.2.8 RESERVATION

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
date_debut	Date	Obligatoire	Date et heure de début de la réservation
date_fin	Date	Obligatoire	Date et heure de fin de la réservation
statut	Enum	Obligatoire	en_attente, confirmee, annulee, terminee
prix_total	Float	Calculé	Montant calculé selon la durée et le tarif de la salle
date_creation	Date	Auto-générée	Date de création de la réservation

2.2.9 PAIEMENT

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
montant	Float	Obligatoire	Montant payé en dollars
date_paiement	Date	Optionnel	Date à laquelle le paiement a été effectué
methode	Enum	Obligatoire	carte, virement, especes
statut	Enum	Obligatoire	en_attente, effectue, rembourse
reference_transaction	String	Unique	Identifiant unique de la transaction bancaire

2.2.10 AVIS

Attribut	Type	Contrainte	Description
_id	ObjectId	Clé primaire	Identifiant unique auto-généré
note	Integer	Obligatoire, 1–5	Note attribuée à la salle
commentaire	String	Optionnel	Texte libre optionnel
date	Date	Auto-générée	Date de publication de l'avis

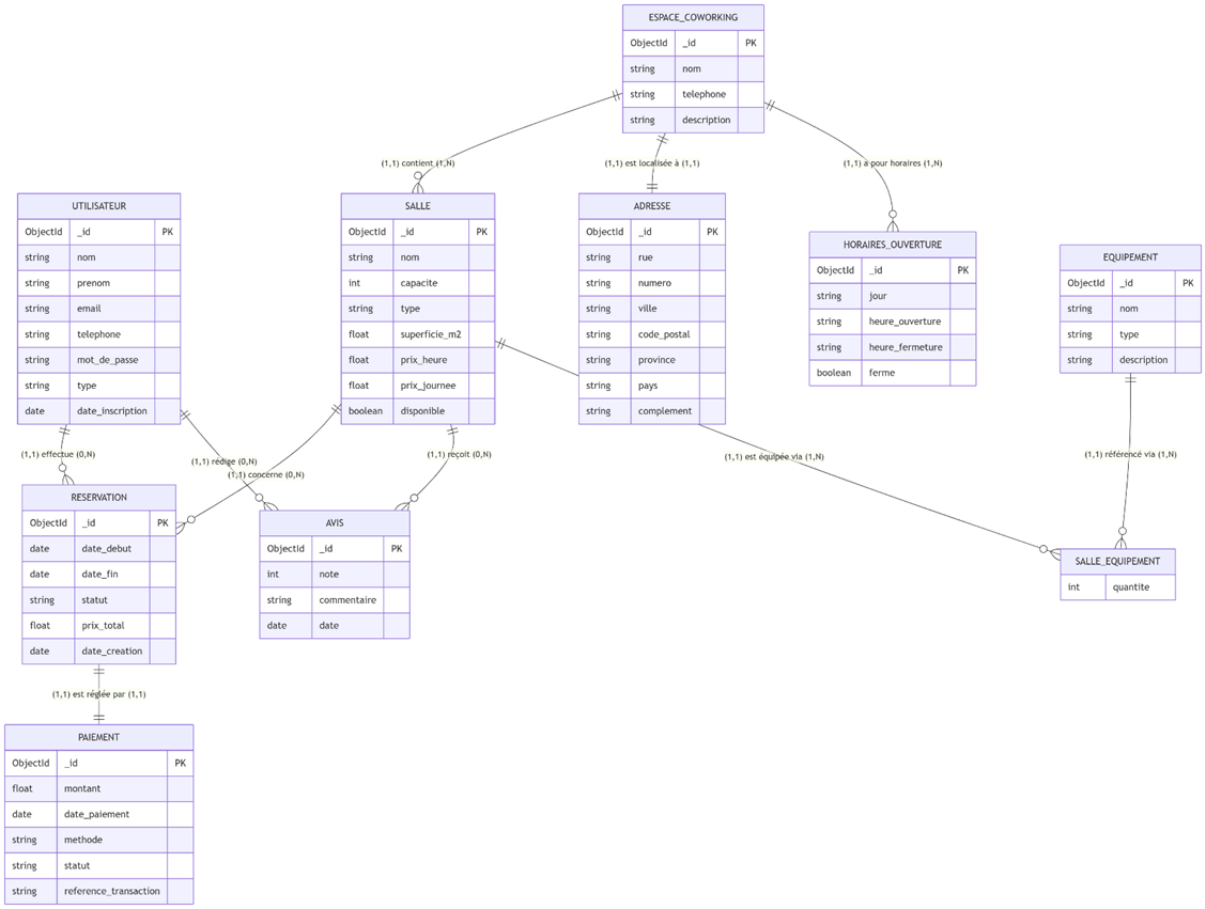
2.3 RELATIONS ET CARDINALITÉS

Le système comporte **10 relations** entre les entités identifiées. Les cardinalités sont exprimées au format **(min, max)** de chaque côté de la relation.

Relation	Entités	Cardinalité	Description
effectue	UTILISATEUR → RESERVATION	(1,1) — (0,N)	Un utilisateur peut effectuer zéro ou plusieurs réservations ; une réservation appartient à exactement un utilisateur
concerne	SALLE → RESERVATION	(1,1) — (0,N)	Une salle peut être concernée par plusieurs réservations ; une réservation porte sur une seule salle
contient	ESPACE_COWORKING → SALLE	(1,1) — (1,N)	Un espace contient au moins une salle ; une salle appartient à un seul espace
est localisée à	ESPACE_COWORKING → ADRESSE	(1,1) — (1,1)	Un espace a exactement une adresse physique ; une adresse correspond à un seul espace
a pour horaires	ESPACE_COWORKING → HORAIRES_OUVERTURE	(1,1) — (1,N)	Un espace possède plusieurs horaires (un par jour) ; chaque horaire appartient à un seul espace
est réglée par	RESERVATION → PAIEMENT	(1,1) — (1,1)	Une réservation génère exactement un paiement ; un paiement est lié à une seule réservation
rédige	UTILISATEUR → AVIS	(1,1) — (0,N)	Un utilisateur peut rédiger zéro ou plusieurs avis ; un avis est rédigé par un seul utilisateur
reçoit	SALLE → AVIS	(1,1) — (0,N)	Une salle peut recevoir plusieurs avis ; un avis porte sur une seule salle
est équipée via	SALLE → SALLE_EQUIPEMENT	(1,1) — (1,N)	Une salle peut avoir plusieurs équipements via l'entité associative
référencé via	EQUIPEMENT → SALLE_EQUIPEMENT	(1,1) — (1,N)	Un équipement peut être présent dans plusieurs salles via l'entité associative

2.4 DIAGRAMME E/A

Le diagramme ci-dessous représente l'ensemble des entités, attributs et relations du système. Les cardinalités sont indiquées directement dans le libellé de chaque relation au format **(min, max)**.



3 ANALYSE DES CHARGES

L'analyse des charges permet d'identifier comment les données seront lues et écrites dans le système, afin de guider les choix de modélisation MongoDB. Cette étape est essentielle pour déterminer quelles structures optimiser en priorité et quels index créer.

3.1 BESOINS FONCTIONNELS DU SYSTÈME

Le système de réservation de salles de coworking repose sur quatre grandes catégories fonctionnelles :

3.1.1 Gestion des utilisateurs

- Inscription et authentification (vérification de l'unicité de l'email)
- Consultation et modification du profil utilisateur
- Consultation de l'historique personnel des réservations

3.1.2 Gestion des salles et espaces

- Recherche de salles selon des critères variables (type, capacité, prix, disponibilité)

- Consultation des détails complets d'une salle (équipements, avis, tarifs)
- Consultation des horaires d'ouverture d'un espace coworking
- Gestion administrative des salles et équipements (ajout, modification, désactivation)

3.1.3 Gestion des réservations

- Vérification de la disponibilité d'un créneau avant réservation
- Création, modification et annulation de réservations
- Consultation de l'historique des réservations par utilisateur ou par salle

3.1.4 Gestion des paiements et avis

- Enregistrement et suivi des paiements liés aux réservations
- Remboursement lors d'une annulation
- Dépôt et consultation des avis sur les salles
- Mise à jour automatique de la note moyenne d'une salle

3.2 Fréquence des requêtes

L'analyse de la fréquence des opérations permet de prioriser les optimisations à apporter à la base de données.

#	Opération	Type	Fréquence estimée	Priorité
1	Recherche de salles disponibles sur un créneau	READ	Très élevée	Critique
2	Consultation des détails d'une salle (équipements, avis)	READ	Très élevée	Critique
3	Vérification de conflit de créneau avant réservation	READ	Élevée	Critique
4	Création d'une réservation	WRITE	Élevée	Critique
5	Consultation du profil utilisateur	READ	Élevée	Haute
6	Consultation de l'historique des réservations	READ	Élevée	Haute
7	Modification ou annulation d'une réservation	UPDATE	Moyenne	Haute
8	Enregistrement et mise à jour d'un paiement	WRITE/UPDATE	Moyenne	Haute
9	Ajout d'un avis et recalcul de la note moyenne	WRITE/UPDATE	Faible	Normale
10	Consultation des avis d'une salle	READ	Faible	Normale
11	Gestion des salles et espaces (admin)	WRITE	Très faible	Basse
12	Gestion des équipements (admin)	WRITE	Très faible	Basse

Les opérations critiques (1, 2, 3, 4) sont celles qui surviennent à chaque visite de la plateforme et doivent être optimisées en priorité via des index et des structures de documents adaptées.

3.3 Volume des données estimé

Le tableau suivant présente une estimation réaliste du volume de données pour une plateforme de coworking dans la province de l'Ontario.

Collection	Volume initial	Volume à 1 an	Volume à 3 ans	Croissance
utilisateurs	500 documents	5 000	15 000	Lente
espaces_coworking	5 documents	20	50	Très lente
salles	50 documents	200	500	Lente
avis	200 documents	10 000	50 000	Moyenne
reservations	1 000 documents	50 000	200 000	**Rapide**

Observation clé : La collection reservations est la plus volumineuse et à la croissance la plus rapide. C'est sur elle que les index et les choix de modélisation auront le plus d'impact sur les performances globales du système.

3.4 Patterns d'accès identifiés

3.4.1 Lecture (READ) — Opérations dominantes

La grande majorité des opérations du système sont des lectures. Trois patterns de lecture ressortent :

- **Accès par salle avec données complètes** : lors de la consultation d'une salle, l'utilisateur a besoin simultanément du nom, de la capacité, du prix, des équipements et des avis récents. Ce pattern favorise l'**embedding** des équipements et d'un sous-ensemble d'avis directement dans le document salle.
- **Accès par créneau horaire** : la vérification de disponibilité filtre systématiquement par salle._id, date_debut et date_fin. Ce pattern impose un **index composite** sur ces trois champs dans la collection reservations.
- **Accès par utilisateur** : l'historique des réservations est consulté par utilisateur._id et trié par date_creation décroissante. Ce pattern impose un **index composite** sur utilisateur._id et date_creation.

3.4.2 Écriture (WRITE) — Opérations sensibles

Les écritures sont moins fréquentes mais plus critiques :

- **Création de réservation** : nécessite une vérification de conflit de créneau en amont, rendant la requête de lecture préalable tout aussi importante que l'écriture elle-même.
- **Mise à jour de la note moyenne** : lors de l'ajout d'un avis, la note moyenne et le sous-ensemble d'avis récents de la salle concernée doivent être recalculés et mis à jour (Computed Pattern + Subset Pattern).
- **Annulation avec remboursement** : la mise à jour simultanée du statut de la réservation et du statut du paiement imbriqué illustre l'avantage de l'embedding du paiement dans la réservation.

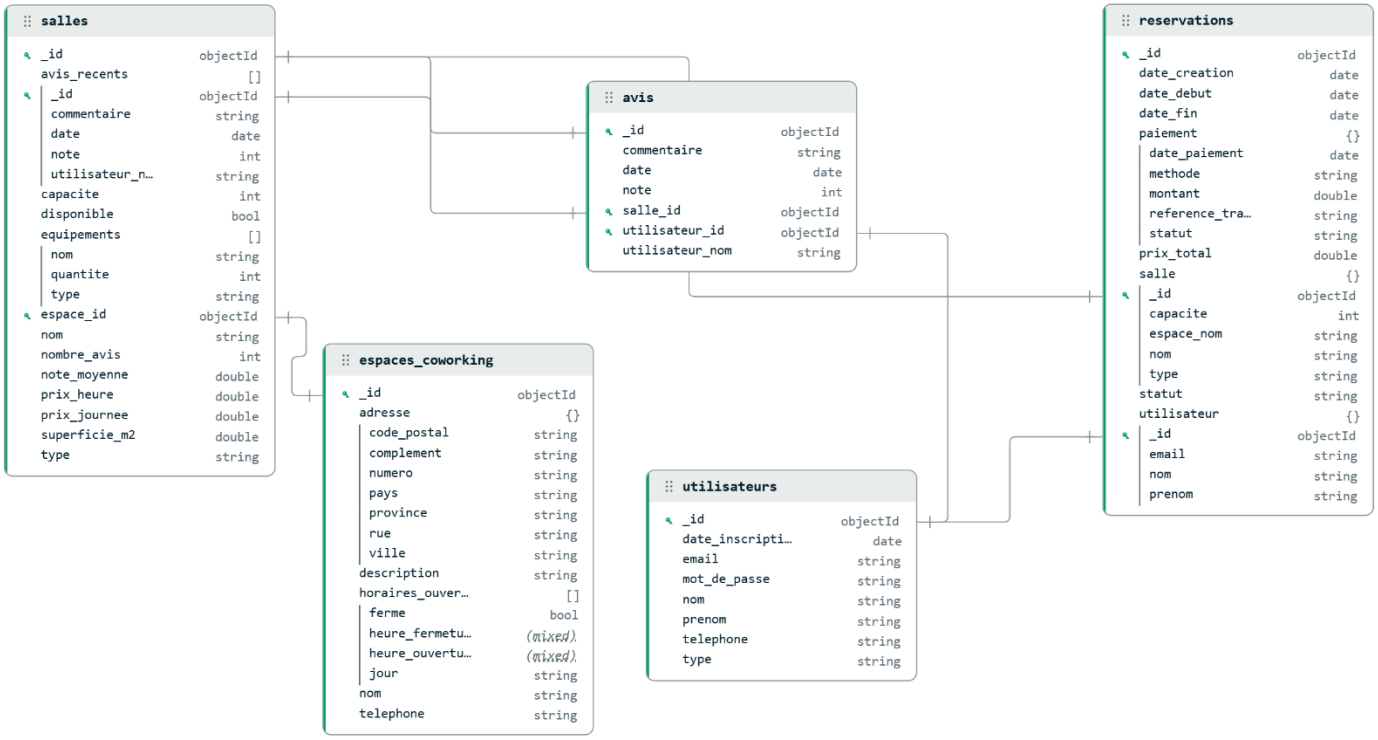
4 Conception — Design Patterns MongoDB

La conception de la base de données MongoDB consiste à traduire le modèle E/A en une structure de documents optimisée. Contrairement aux bases relationnelles, MongoDB ne dispose pas de jointures natives performantes : la structure des documents doit donc être pensée en fonction des patterns d'accès identifiés à l'étape précédente. Chaque choix de modélisation est justifié par les besoins du système.

4.1 Vue d'ensemble des patterns retenus

Relation E/A	Pattern MongoDB retenu	Justification
EspaceCoworking ← Adresse	**Embedding**	Relation 1:1, données toujours lues avec l'espace
EspaceCoworking ← HorairesOuverture	**Embedding**	Données stables, 7 entrées max, accès systématique
Salle ← Equipement	**Embedding**	Liste bornée, toujours affichée avec la salle
Salle ← Avis (derniers)	**Subset Pattern**	Seuls les 5 derniers avis sont imbriqués pour limiter la taille du document
Salle → note moyenne	**Computed Pattern**	Note précalculée pour éviter une agrégation à chaque lecture
Reservation ← Utilisateur / Salle	**Extended Reference Pattern**	Champs fréquemment lus copiés pour éviter les jointures

Reservation ← Paiement	**Embedding**	Relation 1:1, accès toujours conjoint
Utilisateur → Reservations	**Référence**	Volume élevé, requêtes indépendantes
Salle → Avis (complets)	**Collection séparée**	Volume non borné, requêtes indépendantes



4.2 Pattern 1 — Embedding : espaces_coworking

4.2.1 Justification

L'entité Adresse est liée à EspaceCoworking par une relation **1:1** : un espace possède exactement une adresse et celle-ci n'est jamais consultée indépendamment de l'espace. Il en va de même pour HorairesOuverture, dont les 7 entrées (une par jour de semaine) sont systématiquement affichées lors de la consultation d'un espace. Créer des collections séparées pour ces deux entités imposerait des lectures supplémentaires sans aucun bénéfice. L'**Embedding** est donc le choix naturel et optimal pour ces deux relations.

4.2.2 Document JSON

```
{
    _id: ObjectId("..."),
    nom: "CoWork Ottawa ByWard",
    telephone: "+1 613-000-0002",
    description: "Espace de travail moderne au cœur du marché ByWard d'Ottawa",
    adresse: {
        rue: "Rue York",
        numero: "55",
        ville: "Ottawa",
        code_postal: "K1N 5T2",
        province: "Ontario",
        pays: "Canada",
        complement: "Étage 1"
    },
    horaires_ouverture: [
        { jour: "lundi", heure_ouverture: "07:00", heure_fermeture: "21:00", ferme: false },
        { jour: "mardi", heure_ouverture: "07:00", heure_fermeture: "21:00", ferme: false },
        { jour: "mercredi", heure_ouverture: "07:00", heure_fermeture: "21:00", ferme: false },
        { jour: "jeudi", heure_ouverture: "07:00", heure_fermeture: "21:00", ferme: false },
        { jour: "vendredi", heure_ouverture: "07:00", heure_fermeture: "19:00", ferme: false },
```

```
        { jour: "samedi", heure_ouverture: "09:00", heure_fermeture: "17:00", ferme: false },
        { jour: "dimanche", heure_ouverture: null, heure_fermeture: null, ferme: true }
    ]
}
```

4.3 Pattern 2 — Embedding + Subset + Computed : salles

4.3.1 Justification

La collection salles concentre trois patterns distincts :

- **Embedding des équipements** : la liste des équipements d'une salle est toujours affichée avec elle, est de taille bornée (rarement plus de 10 équipements) et n'est jamais interrogée de manière indépendante. L'embedding évite une jointure systématique.
- **Subset Pattern pour les avis** : une salle peut recevoir des centaines d'avis au fil du temps. Imbriquer la totalité des avis ferait grossir le document indéfiniment, dégradant les performances de lecture. On imbrique uniquement les **5 derniers avis** pour l'affichage immédiat, tandis que l'historique complet est conservé dans la collection avis.
- **Computed Pattern pour la note moyenne** : calculer la note moyenne à chaque lecture de la salle nécessiterait une agrégation coûteuse sur la collection avis. On stocke à la place la valeur précalculée note_moyenne directement dans le document salle, mise à jour uniquement lors de l'ajout d'un nouvel avis.

4.3.2 Document JSON

```
{
    _id: ObjectId("..."),
    espace_id: espace1.insertedId,
    nom: "Salle Boréale",
    capacite: 8,
    type: "reunion",
    superficie_m2: Double(25),
    prix_heure: Double(35),
    prix_journee: Double(210),
    disponible: true,
    note_moyenne: 4.3,
    nombre_avis: 12,
    equipements: [
        { nom: "Projecteur HD", type: "technologique", quantite: 1 },
        { nom: "Tableau blanc", type: "mobilier", quantite: 2 },
        { nom: "Vidéoconférence", type: "technologique", quantite: 1 },
        { nom: "Climatisation", type: "service", quantite: 1 }
    ],
    avis_recents: [
        {
            utilisateur_nom: "Channel T.",
            note: 5,
            commentaire: "Salle très bien équipée, parfaite pour nos réunions d'équipe.",
            date: ISODate("2026-03-15T14:30:00Z")
        },
        {
            utilisateur_nom: "Josue M.",
            note: 4,
            commentaire: "Bonne salle, le projecteur est un peu vieux mais fonctionnel.",
            date: ISODate("2026-03-10T10:00:00Z")
        }
    ]
}
```

4.4 Pattern 3 — Extended Reference : reservations

4.4.1 Justification

La collection reservations référence deux entités volumineuses : Utilisateur et Salle. Une référence pure obligerait à effectuer deux lectures supplémentaires à chaque affichage de l'historique des réservations. L'**Extended Reference Pattern** consiste à copier les champs les plus fréquemment consultés (nom, prenom, email pour l'utilisateur ; nom, type, capacite, espace_nom pour la salle) directement dans le document réservation. Les identifiants originaux sont conservés pour permettre des requêtes approfondies si nécessaire.

De plus, le **Paiement** est directement imbriqué dans la réservation car la relation est **1:1** et les deux sont systématiquement consultés ensemble (notamment pour vérifier le statut de paiement lors d'une annulation).

4.4.2 Document JSON

```
{
    date_debut: ISODate("2026-04-01T09:00:00Z"),
    date_fin: ISODate("2026-04-01T12:00:00Z"),
    statut: "confirmee",
    prix_total: Double(105),
    date_creation: ISODate("2026-03-19T10:00:00Z"),
    utilisateur: {
        _id: user1.insertedId,
        nom: "Talling",
        prenom: "Channel",
        email: "2751491@coworking.ca"
    },
    salle: {
        _id: salle1.insertedId,
        nom: "Salle Boréale",
        type: "reunion",
        capacite: 8,
        espace_nom: "CoWork Québec Vieux-Port"
    },
    paiement: {
        montant: Double(105),
        date_paiement: ISODate("2026-03-19T10:05:00Z"),
        methode: "carte",
        statut: "effectue",
        reference_transaction: "TXN-20260401-0001"
    }
}
```

4.5 Pattern 4 — Collections séparées : utilisateurs et avis

4.5.1 Justification

- **Utilisateurs** : les données utilisateur sont consultées de manière indépendante (connexion, gestion de profil) et référencées par de nombreuses réservations. Une collection dédiée avec un index unique sur email est le choix naturel.
- **Avis** : bien que les 5 derniers avis soient imbriqués dans la salle via le Subset Pattern, l'historique complet des avis doit être conservé dans une collection séparée pour deux raisons : leur volume est **non borné** dans le temps, et ils peuvent être consultés indépendamment (ex : tous les avis d'un utilisateur). Chaque avis duplique le nom de l'utilisateur via une **Extended Reference** légère pour éviter une jointure lors de l'affichage.

4.5.2 Documents JSON

```
// Collection Utilisateur
{
    _id: ObjectId("..."),
    nom: "Misse",
```

```
        prenom: "Anthony",
        email: "2753251@coworking.ca",
        telephone: "+1 613-555-6666",
        mot_de_passe: "$2b$10$hashedpassword3",
        type: "admin",
        date_inscription: ISODate("2025-01-10T00:00:00Z")
    }


    // Collection Avis
    {
        _id: ObjectId("..."),
        salle_id:        ObjectId("..."),
        utilisateur_id:  ObjectId("..."),
        utilisateur_nom: "Josue M.",
        note: 5,
        commentaire: "Bureau calme et très bien installé, je reviendrai.",
        date: ISODate("2026-03-12T16:00:00Z")
    }
```

4.6 Schéma final des collections

Le modèle final de la base de données coworking_db se compose de **5 collections**, chacune optimisée selon les patterns identifiés :

coworking_db

- ***espaces_coworking***
  - *Embedding : adresse (objet imbriqué)*
  - *Embedding : horaires_ouverture (tableau de 7 objets)*
- ***salles***
  - *Référence vers espaces_coworking (espace_id)*
  - *Embedding : equipements (tableau borné)*
  - *Subset Pattern : avis_recents (5 derniers avis)*
  - *Computed Pattern : note_moyenne + nombre_avis*
- ***reservations***
  - *Extended Reference : utilisateur (champs copiés + _id)*
  - *Extended Reference : salle (champs copiés + _id)*
  - *Embedding : paiement (objet imbriqué 1:1)*
- ***utilisateurs***
  - *Document simple + index unique sur email*
- ***avis***
  - *Référence vers salles (salle_id)*
  - *Référence vers utilisateurs (utilisateur_id)*
  - *Extended Reference légère : utilisateur_nom (champ copié)*

Ce schéma permet de répondre aux requêtes les plus fréquentes — consultation de salle, vérification de créneau, historique utilisateur — **en une seule lecture**, sans jointure applicative, conformément aux principes fondamentaux de la modélisation MongoDB.

5 Implémentation

Cette section présente la mise en place concrète de la base de données coworking_db dans MongoDB, en trois étapes : la création des collections avec validation, la création des index d'optimisation, et l'insertion des données de test.

5.1 Création de la base de données et des collections

La base de données coworking_db est créée automatiquement par MongoDB lors de la première commande use. Chaque collection est ensuite créée avec un validateur $jsonSchema qui garantit l'intégrité des données en imposant des types et des champs obligatoires sur chaque document inséré.

5.1.1 Sélection de la base de données

```
>_MONGOSH
> use coworking_db
< switched to db coworking_db
```

5.1.2 Création des collections

```
> db.getCollectionNames()
< [
    'espaces_coworking',
    'reservations',
    'utilisateurs',
    'salles',
    'avis'
  ]
```

Explication : La validation $jsonSchema fonctionne comme suit :

- bsonType: "object" — chaque document doit être un objet JSON
- required: [...] — liste des champs obligatoires à la création
- properties: {...} — définition du type attendu pour chaque champ
- Tout document ne respectant pas ces règles est **automatiquement rejeté** par MongoDB avec une erreur Document failed validation

5.2 Insertion des données de test

Les données de test ont été insérées via le fichier 03_insertion.mongodb.js. Les variables (user1, espace1, salle1…) permettent de récupérer automatiquement les ObjectId générés pour les utiliser dans les documents liés (Extended Reference Pattern).

5.2.1 Vérification du nombre de documents insérés

```
> db.utilisateurs.countDocuments()
< 4
> db.espaces_coworking.countDocuments()
< 2
> db.salles.countDocuments()
< 3
> db.reservations.countDocuments()
< 4
> db.avis.countDocuments()
< 5
```

5.2.2 Visualisation d'un document salle complet

```
> db.salles.findOne({ nom: "Salle Boréale" })
< {
    _id: ObjectId('69bc19a6a8e8ddd677998365'),
    espace_id: ObjectId('69bc19a6a8e8ddd677998363'),
    nom: 'Salle Boréale',
    capacite: 8,
    type: 'reunion',
    superficie_m2: 25,
    prix_heure: 35,
    prix_journee: 210,
    disponible: true,
    note_moyenne: 4.666666666666667,
    nombre_avis: 3,
    equipements: [
      {
        nom: 'Projecteur HD',
        type: 'technologique',
        quantite: 1
      },
      {
        nom: 'Tableau blanc',
        type: 'mobilier',
        quantite: 2
      },
      {
        nom: 'Vidéoconférence',
        type: 'technologique',
        quantite: 1
      },
      {
        nom: 'Climatisation',
        type: 'service',
        quantite: 1
      }
    ],
    avis_recents: [
      {
        _id: ObjectId('69bc19ac61981b861e7ef4b0'),
        utilisateur_nom: 'Anthony M.',
        note: 5,
        commentaire: 'Excellent espace, très professionnel et bien équipé.',
        date: 2026-03-19T15:43:40.270Z
      },
      {
        _id: ObjectId('69bc19a6a8e8ddd67799836b'),
        utilisateur_nom: 'Channel T.',
        note: 5,
        commentaire: "Salle très bien équipée, parfaite pour nos réunions d'équipe.",
        date: 2026-03-15T14:30:00.000Z
      },
      {
        _id: ObjectId('69bc19a6a8e8ddd67799836c'),
        utilisateur_nom: 'Josue M.',
        note: 4,
        commentaire: 'Bonne salle, le projecteur est un peu vieux mais fonctionnel.',
        date: 2026-03-10T10:00:00.000Z
      }
    ]
  }
```

Cette capture illustre concrètement l'**Embedding** des équipements, le **Subset Pattern** des avis récents et le **Computed Pattern** de la note moyenne dans un seul document.

5.3 Vue des collections dans MongoDB Compass

localhost:27017 > coworking_db

Open MongoDB shell | Create collection | Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
avis	-	20.48 kB	817.00 B	4	204.00 B	3	61.44 kB
espaces_coworking	-	20.48 kB	1.87 kB	2	933.00 B	2	40.96 kB
reservations	-	20.48 kB	1.50 kB	3	501.00 B	4	81.92 kB
salles	-	20.48 kB	1.45 kB	3	482.00 B	4	81.92 kB
utilisateurs	-	20.48 kB	817.00 B	4	204.00 B	2	40.96 kB

6 Exécution des requêtes CRUD

Les cinq requêtes suivantes ont été rédigées et exécutées afin de valider le bon fonctionnement de la base de données et de démontrer concrètement les design patterns appliqués.

6.1 Requête 1 — Recherche de salles disponibles

6.1.1 Objectif

Retrouver toutes les salles disponibles ayant une capacité minimale de 5 personnes, triées par prix horaire croissant. Cette requête simule la recherche d'une salle par un utilisateur depuis l'interface de la plateforme.

6.1.2 Pattern mis en valeur

Index composite sur (type, capacite, prix_heure) — la requête est résolue sans parcourir l'ensemble de la collection grâce à l'index créé à l'étape 5.2.

```
> db.salles.find({ disponible: true, capacite: { $gte: 5 } }, { nom: 1, type: 1, capacite: 1, prix_heure: 1, note_moyenne: 1 }).sort({ prix_heure: 1 })
< {
    _id: ObjectId('69bc19a6a8e8ddd677998367'),
    nom: 'Le Hub Capital',
    capacite: 20,
    type: 'espace_ouvert',
    prix_heure: 15,
    note_moyenne: 4.1
  }
  {
    _id: ObjectId('69bc19a6a8e8ddd677998365'),
    nom: 'Salle Boréale',
    capacite: 8,
    type: 'reunion',
    prix_heure: 35,
    note_moyenne: 4.666666666666667
  }
```

6.1.3 Analyse

La requête retourne uniquement les salles dont la capacité est supérieure ou égale à 5 et dont le statut disponible est true. La projection limite les champs retournés aux informations utiles pour l'affichage, réduisant la quantité de données transmises. Le tri par prix_heure croissant permet à l'utilisateur de voir les options les moins chères en premier.

6.2 Requête 2 — Vérification de créneau et création de réservation

6.2.1 Objectif

Vérifier qu'aucune réservation active n'existe pour une salle donnée sur un créneau donné, puis créer la réservation si le créneau est libre. Cette requête est la plus critique du système car elle conditionne toute nouvelle réservation.

6.2.2 Pattern mis en valeur

Index composite sur (salle._id, date_debut, date_fin) pour la détection de conflit. **Extended Reference Pattern** pour copier les champs de l'utilisateur et de la salle dans la réservation. **Embedding** du paiement dans la réservation.

```
// Étape 1 — Vérification du conflit de créneau
const salleId = db.salles.findOne({ nom: "Salle Boréale" })._id;

const conflit = db.reservations.findOne({
    "salle._id": salleId,
    statut: { $in: ["en_attente", "confirmee"] },
    date_debut: { $lt: ISODate("2026-04-10T14:00:00Z") },
    date_fin: { $gt: ISODate("2026-04-10T10:00:00Z") }
});

// Étape 2 — Insertion seulement si pas de conflit
if (!conflit) {
    const salle = db.salles.findOne({ nom: "Salle Boréale" });
    const user = db.utilisateurs.findOne({ email: "2753251@coworking.ca" });

    db.reservations.insertOne({
        date_debut: ISODate("2026-04-10T10:00:00Z"),
        date_fin: ISODate("2026-04-10T14:00:00Z"),
        statut: "confirmee",
        prix_total: Double(salle.prix_heure * 4),
        date_creation: new Date(),
        utilisateur: {
            _id: user._id,
            nom: user.nom,
            prenom: user.prenom,
            email: user.email
        },
        salle: {
            _id: salle._id,
            nom: salle.nom,
            type: salle.type,
            capacite: salle.capacite,
            espace_nom: "CoWork Québec Vieux-Port"
        },
        paiement: {
            montant: Double(salle.prix_heure * 4),
            date_paiement: new Date(),
            methode: "carte",
            statut: "effectue",
            reference_transaction: "TXN-20260410-0004"
        }
    });
    print("Réservation créée avec succès.");
} else {
    print("Créneau non disponible — conflit détecté.");
}
```

6.2.3 Analyse

La logique de détection repose sur le chevauchement de créneaux : un conflit existe si date_debut_existante < date_fin_nouvelle ET date_fin_existante > date_debut_nouvelle. Seules les réservations en_attente ou confirmee sont considérées, les réservations annulee ou terminee libérant le créneau. Si un conflit est détecté lors d'une seconde exécution, cela confirme que la première réservation a bien été créée et que la protection fonctionne correctement.

6.3 Requête 3 — Annulation d'une réservation avec remboursement

6.3.1 Objectif

Modifier simultanément le statut d'une réservation à annulee et le statut de son paiement imbriqué à rembourse, en une seule opération atomique.

6.3.2 Pattern mis en valeur

Embedding du paiement dans la réservation : la mise à jour de deux champs appartenant au même document est atomique dans MongoDB, garantissant qu'il n'existe pas d'état incohérent où une réservation serait annulée sans que son paiement soit remboursé.

```
> db.reservations.findOne({ "paiement.reference_transaction": "TXN-20260402-0002" }, { statut: 1, "paiement.statut": 1 })
< {
    _id: ObjectId('69bc19a6a8e8ddd677998369'),
    statut: 'confirmee',
    paiement: {
      statut: 'effectue'
    }
  }
```

6.3.3 Analyse

La notation pointée ("paiement.statut") permet d'accéder directement à un champ d'un sous-document imbriqué sans avoir à réécrire l'objet paiement entier. Cette opération démontre l'avantage clé de l'embedding : les deux mises à jour sont effectuées en **une seule opération atomique** au niveau du document, ce qui serait impossible avec deux tables séparées en SQL sans transaction explicite.

6.4 Requête 4 — Ajout d'un avis et mise à jour de la salle

6.4.1 Objectif

Insérer un nouvel avis dans la collection avis, recalculer la note moyenne de la salle concernée, et mettre à jour le sous-ensemble des 5 avis récents directement dans le document salle.

6.4.2 Pattern mis en valeur

Subset Pattern : seuls les 5 derniers avis sont maintenus dans le document salle. **Computed Pattern** : la note moyenne est recalculée et stockée directement, évitant une agrégation à chaque lecture.

```
> db.salles.findOne({ nom: "Salle Boréale" }, { note_moyenne: 1, nombre_avis: 1, avis_recents: 1 })
< {
    _id: ObjectId('69bc19a6a8e8ddd677998365'),
    note_moyenne: 4.666666666666667,
    nombre_avis: 3,
    avis_recents: [
      {
        _id: ObjectId('69bc19ac61981b861e7ef4b0'),
        utilisateur_nom: 'Anthony M.',
        note: 5,
        commentaire: 'Excellent espace, très professionnel et bien équipé.',
        date: 2026-03-19T15:43:40.270Z
      },
      {
        _id: ObjectId('69bc19a6a8e8ddd67799836b'),
        utilisateur_nom: 'Channel T.',
        note: 5,
        commentaire: "Salle très bien équipée, parfaite pour nos réunions d'équipe.",
        date: 2026-03-15T14:30:00.000Z
      },
      {
        _id: ObjectId('69bc19a6a8e8ddd67799836c'),
        utilisateur_nom: 'Josue M.',
        note: 4,
        commentaire: 'Bonne salle, le projecteur est un peu vieux mais fonctionnel.',
        date: 2026-03-10T10:00:00.000Z
      }
    ]
```

6.4.3 Analyse

Cette requête illustre le cycle complet de mise à jour des patterns Computed et Subset. La note moyenne passe de 4.3 à une valeur recalculée intégrant le nouvel avis. Le tableau avis_recents est remplacé par les 5 avis les plus récents triés par date décroissante. Ce mécanisme garantit que la lecture d'une salle reste toujours rapide, sans agrégation à la volée.

6.5 Requête 5 — Historique des réservations d'un utilisateur

6.5.1 Objectif

Retrouver l'ensemble des réservations d'un utilisateur avec ses statistiques globales : nombre total de réservations, nombre par statut et montant total dépensé.

6.5.2 Pattern mis en valeur

Extended Reference Pattern : les champs utilisateur.nom, utilisateur.prenom et utilisateur.email copiés dans chaque réservation permettent de filtrer et d'afficher l'historique sans consulter la collection utilisateurs. **Aggregation Pipeline** pour les statistiques.

```
> db.reservations.aggregate([{ $match: { "utilisateur.email": "2751491@coworking.ca" } }, { $sort: { date_
< {
    _id: ObjectId('69bc19a6a8e8ddd67799835f'),
    nom_complet: 'Channel Talling',
    total_depense: 195,
    nb_total: 2,
    nb_confirmees: 1,
    nb_annulees: 1
  }
```

6.5.3 Analyse

Le pipeline se déroule en quatre étapes : $match filtre par email, $sort ordonne par date décroissante, $group agrège les statistiques, et $project formate le résultat final. Le résultat obtenu pour Channel Talling montre 2 réservations (1 confirmée, 1 annulée) pour un total de 195 $, ce qui correspond exactement aux données insérées et valide le bon fonctionnement de l'Extended Reference Pattern.

7 Conclusion

7.1 Bilan des choix de modélisation

Ce projet a permis de concevoir et d'implémenter une base de données MongoDB complète pour un système de réservation de salles de coworking. La démarche suivie — modélisation E/A, analyse des charges, conception par patterns, implémentation et requêtes — a montré l'importance de raisonner en fonction des **patterns d'accès** plutôt qu'en fonction des règles de normalisation classiques.

Les principaux choix de modélisation retenus sont les suivants :

- **L'Embedding** de l'adresse, des horaires et des équipements permet de récupérer toutes les informations d'un espace ou d'une salle en **une seule lecture**, sans jointure applicative
- **Le Subset Pattern** appliqué aux avis évite la croissance illimitée des documents salles tout en offrant un affichage immédiat des avis les plus récents
- **Le Computed Pattern** sur la note moyenne élimine toute agrégation coûteuse à la lecture, au prix d'une mise à jour légère à chaque nouvel avis
- **L'Extended Reference Pattern** dans les réservations garantit l'autonomie des documents et la rapidité de l'historique utilisateur, sans dépendance aux collections utilisateurs ou salles
- **L'Embedding du paiement** dans la réservation assure l'atomicité de l'annulation et du remboursement en une seule opération

7.2 Avantages de MongoDB par rapport à un modèle relationnel

Pour ce projet spécifique, MongoDB présente plusieurs avantages significatifs par rapport à un SGBD relationnel comme MySQL ou PostgreSQL :

Critère	Modèle relationnel	MongoDB
Structure des données	Tables rigides, schéma prédéfini	Documents JSON flexibles, schéma dynamique

Requêtes fréquentes	Jointures multiples (salles + équipements + avis)	Lecture d'un seul document (embedding)
Évolutivité	Migration de schéma nécessaire	Ajout de champs sans impact sur les autres documents
Hiérarchie naturelle	Décomposée en plusieurs tables	Représentée nativement (horaires dans espace, paiement dans réservation)
Scalabilité	Verticale (serveur plus puissant)	Horizontale (sharding sur plusieurs serveurs)
Performances en lecture	Dégradées par les JOIN	Optimisées par l'embedding et les index composites

La nature hiérarchique des données de ce projet — un espace contient des salles, qui ont des équipements et des avis, une réservation encapsule un paiement — se modélise **naturellement en documents imbriqués**, ce qui justifie pleinement le choix de MongoDB.

7.3 LIMITES ET AMÉLIORATIONS POSSIBLES

Malgré les choix effectués, plusieurs limites ont été identifiées et pourraient faire l'objet d'améliorations futures :

7.3.1 Gestion des conflits de créneaux

La vérification de disponibilité est actuellement effectuée côté application, avant l'insertion. Dans un environnement à forte concurrence (plusieurs utilisateurs réservant simultanément), cette approche n'est pas atomique et pourrait entraîner des doubles réservations. Une amélioration consisterait à utiliser les **transactions multi-documents** de MongoDB pour garantir l'atomicité de la vérification et de l'insertion.

7.3.2 Gestion des abonnements

Le modèle actuel gère uniquement des réservations à l'unité. Une évolution naturelle serait d'ajouter une entité Abonnement permettant aux membres réguliers de bénéficier de tarifs préférentiels ou d'un accès illimité à certaines salles sur une période donnée.

7.3.3 Synchronisation du Computed Pattern

La note moyenne est mise à jour manuellement après chaque avis. Dans un système de production, cette synchronisation devrait être automatisée via un **trigger** ou un mécanisme de mise à jour en arrière-plan pour éviter des valeurs temporairement désynchronisées.

7.3.4 Données dupliquées dans l'Extended Reference

La duplication des champs utilisateur et salle dans les réservations implique qu'une modification du nom d'un utilisateur ou d'une salle ne se propage pas automatiquement aux réservations existantes. Ce compromis, acceptable pour des données historiques, doit être documenté et géré explicitement dans l'application.